

The Ultimate Web Development Handbook



What is the DOM?

Introduction

The **DOM (Document Object Model)** is a programming interface provided by the browser that represents an HTML or XML document as a structured tree of objects. Each element, attribute, and piece of text in the HTML document becomes a **node** in the DOM tree.

This allows JavaScript to interact with the HTML and CSS of a web page — you can use JavaScript to read and modify the page's structure, content, and style dynamically.

Key Points:

- The DOM is **not** part of JavaScript, but it is provided by the browser's **Web APIs**.
- The browser turns HTML into a tree structure called the DOM.
- JavaScript can use this tree to:
 - Select elements
 - Modify content or style
 - Add or remove elements
 - Respond to user interactions

Example HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>DOM Example</title>
  </head>
  <body>
```

```
<h1>Hello, DOM!</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

How the DOM Sees It

The browser parses the above HTML and creates a tree-like structure:

- document
 - html
 - head
 - title
 - body
 - h1
 - p

Accessing the DOM in JavaScript

The `document` object is the entry point to the DOM in JavaScript.

```
console.log(document); // Logs the entire DOM tree
console.log(document.body); // Logs the <body> element
console.log(document.title); // Logs the content of <title>
```

Summary

- The DOM represents the page so that JavaScript can interact with it.
- You can access and modify HTML elements using JavaScript through the DOM.
- Understanding the DOM is essential for web development and dynamic page interactions.

Accessing the DOM

Introduction

To manipulate or interact with elements in an HTML document using JavaScript, you must first **access the DOM**. The browser provides two main global objects for this:

- `window`
 - `document`
-

The `window` Object

The `window` object represents the browser window. It is the **global scope** in a browser environment. All global variables and functions become properties of the `window` object.

```
console.log(window); // Logs the global window object
console.log(window.innerWidth); // Gets the width of the browser window
```

You don't usually need to reference `window` explicitly, because it's the default context:

```
alert("Hello"); // Same as window.alert("Hello")
```

The document Object

The `document` object is a property of the `window` and serves as the main entry point to the web page's DOM.

```
console.log(document); // Logs the entire HTML document as a DOM tree
console.log(document.URL); // Gets the current page URL
```

Common Properties of document

Property	Description
<code>document.body</code>	Refers to the <code><body></code> element
<code>document.head</code>	Refers to the <code><head></code> element
<code>document.title</code>	Gets or sets the document's title
<code>document.URL</code>	Returns the full URL of the page

Example

```
<!DOCTYPE html>
<html>
<head>
  <title>Accessing the DOM</title>
</head>
<body>
  <h1>Hello World</h1>
  <script>
    console.log(document.title); // "Accessing the DOM"
    document.title = "New Title";
  </script>
</body>
</html>
```

Summary

- The `window` object is the global context and represents the browser window.
- The `document` object gives you access to the DOM structure of the HTML page.
- You use `document` to navigate and manipulate HTML elements via JavaScript.

Selecting Elements in JavaScript

When working with the DOM (Document Object Model), selecting elements is often the first step to manipulate them. JavaScript provides multiple methods to select HTML elements based on their ID, class, tag name, or CSS selector.

1. `getElementById`

Selects a single element by its unique `id`.

2. `getElementsByClassName`

Returns a live `HTMLCollection` of all elements with the specified class name.

3. `getElementsByTagName`

Returns a live `HTMLCollection` of all elements with the specified tag name (e.g., `div`, `p`, `h1`, etc.).

4. `querySelector`

Returns the first element that matches a specified CSS selector.

5. `querySelectorAll`

Returns a static `NodeList` of all elements that match a specified CSS selector.

These methods allow you to access and manipulate elements dynamically. We'll explore each of them in detail with examples next.